

tensormodels: Statistical Models for Tensor-Valued Data

by Anupama Kannan, Dr. David Kahle, and Dr. Michael Gallagher

Abstract *tensormodels* is an R package for simulation, maximum likelihood estimation, and model diagnostics for tensor-variate distributions arising in multiway array data. The package implements the tensor-variate normal distribution and several normal variance–mean mixture families, including the skew-t, generalized hyperbolic, variance gamma, and normal inverse Gaussian distributions. Estimation is performed via likelihood-based and expectation–maximization algorithms under a Kronecker-separable covariance model, yielding mode-specific covariance estimates through iterative updates. Model assessment tools include distance-based diagnostics, such as Mahalanobis distance plots constructed from tensor-whitened residuals. In addition, *tensormodels* provides implementations of widely used tensor decompositions for approximation and dimension reduction, including Tensor-Train decomposition via TT-SVD, Higher-Order Singular Value Decomposition (HOSVD), and Parallel Factor Analysis (CP/PARAFAC) computed using alternating least squares. It includes common linear algebra and tensor operations, such as matricization, the n-mode products, and the Khatri-Rao product.

1 Introduction

Lately, with the rise of data measurement tools and big data, the creation of higher dimensional data has become more and more common. These unique types of data require corresponding unique analytical tools to properly fit models with interpretation. Many researchers default to vectorizing or matricizing this higher order data to fit models that they are comfortable with and have pre-established tools. Additionally, they will commonly assume that data satisfies the assumptions of normality, which can sometimes be a very extreme choice that is only made to make computation and analysis easier.

The package **tensormodels** aims to assist with both of these issues. Firstly, it contains the tensor variate normal distribution and several skewed distributions under the normal mean-variance family model that can model a multitude of different shapes of data. Secondly, it contains diagnostic test and model heuristics to pick the best distribution for the user’s data set. It also contains random number generation and density functions for researchers who merely want to better understand tensor variate distributions.

In the following sections, we will explain the theory behind tensor variate distributions, explain some unique choices in the package design, show a simulation set up, and apply the models to a real-world problem.

Explain the statistical or computational problem that motivates **tensormodels**. State who the package is for and summarize its main contributions. End this section with a short roadmap of the paper.

2 Background and related work

The tensor space is still being established, with works using conflicting notation. In this section, we will set a standard for certain operations, and explain the basic underlying theory for several of the functions. Tensors are a generalization of vectors and matrices, allowing us to represent higher-order data into a structure. We need new terminology to describe these objects.

Order

The order of a tensor is the number of indices needed to specify a value. A vector has order 1 and a matrix has order 2, while the order of a tensor is 3 or more. Another way to define the order is the number of values needed in R’s subsetting operator to pull out a specific element.

For example, take the vector $\mathbf{a} = (1, 2, 3)$. To extract a specific element, you only require one value, so it has an order 1.

```
a <- c(1, 2, 3)
```

```
a[2]
```

```
#> [1] 2
```

Now consider the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

In R, we would need to specify two indices to pull out an element - the first represents the row, and the second represents the column.

```
A <- matrix(data = 1:6, nrow = 2, ncol = 3)
```

```
A[2, 1]
#> [1] 2
```

Consider a third-order tensor. In R, we would need to specify three indices to pull out an element.

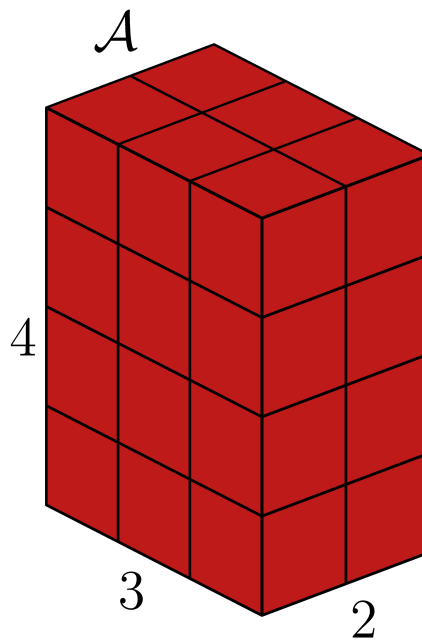


Figure 1: A $4 \times 3 \times 2$ tensor $\mathcal{A}$

```
A <- array(data = 1:24, dim = c(4, 3, 2))
```

```
A[3, 1, 1]
#> [1] 3
```

Mode

The mode of an object is its specific axis. For a matrix, it is common terminology to refer to the first mode as the rows and the second mode as the columns. However, since a tensor can have three or more dimensions, we now need a term for the specific dimension being referred to.

Consider a tensor $\mathcal{A} \in \mathbb{R}^{4 \times 2 \times 3}$. We would say the first mode is 4, the second mode is 2, and the third mode is 3. Note that the modes of a tensor are dependent on the structure of the object. For example, we could keep the same data in $\mathcal{A}$ but permute the modes, leading to a new tensor $\mathcal{B} \in \mathbb{R}^{4 \times 3 \times 2}$. When communicating a tensor, it is important to keep the modes consistent and keep track of what each mode represents.

n -mode product

The n -mode matrix product of a tensor $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_n}$ with a matrix $\mathbf{U} \in \mathbb{R}^{J \times I_n}$ is

$$(\mathcal{A} \times_n \mathbf{U}) \in \mathbb{R}^{I_1 \times \dots \times I_{n-1} \times J \times I_{n+1} \times \dots \times I_N}$$

with entries

$$(\mathcal{A} \times_n \mathbf{U})_{i_1, \dots, i_{n-1}, j, i_{n+1}, \dots, i_N} = \sum_{i_n=1}^{I_n} a_{i_1, i_2, \dots, i_N} u_{j, i_n}.$$

The tensor and matrix share one mode in common, denoted here as I_n . This operation is also called the tensor times matrix product.

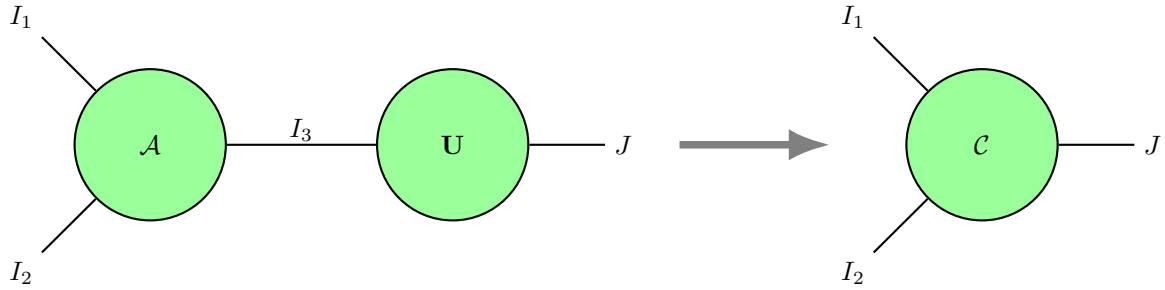


Figure 2: n -mode product

Example

Let $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \in {}^3$ and $\mathbf{U} = \begin{bmatrix} 4 & 6 & 8 \\ 5 & 7 & 9 \end{bmatrix} \in {}^{2 \times 3}$.

$$\mathbf{a} \times_1 \mathbf{U} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \times_1 \begin{bmatrix} 4 & 6 & 8 \\ 5 & 7 & 9 \end{bmatrix} = \begin{bmatrix} 1(4) + 2(6) + 3(8) \\ 1(5) + 2(7) + 3(9) \end{bmatrix} = \begin{bmatrix} 40 \\ 46 \end{bmatrix}$$

nm -mode product

The ${}_n \times_m$ -product of a tensor $\mathcal{A} \in {}^{I_1 \times I_2 \times \dots \times I_N}$ with a tensor $\mathcal{B} \in {}^{J_1 \times J_2 \times \dots \times J_M}$ such that $I_n = J_m$, is defined as $\mathcal{C} = \mathcal{A} \times_n \mathcal{B}$. It has entries

$$\mathcal{C}(i_1, \dots, i_{n-1}, \dots, i_N, j_1, \dots, j_{m-1}, j_{m+1}, \dots, j_M) = \sum_{i_n=1}^{I_n} \mathcal{A}(i_1, \dots, i_{n-1}, i_n, i_{n+1}, \dots, i_N) \mathcal{B}(j_1, \dots, j_{m-1}, i_n, j_{m+1}, \dots, j_M)$$

Note that this operation is different from the tensor product. It is like the n -mode product, except now a mode m also has to be specified since we are indexing over two tensors.

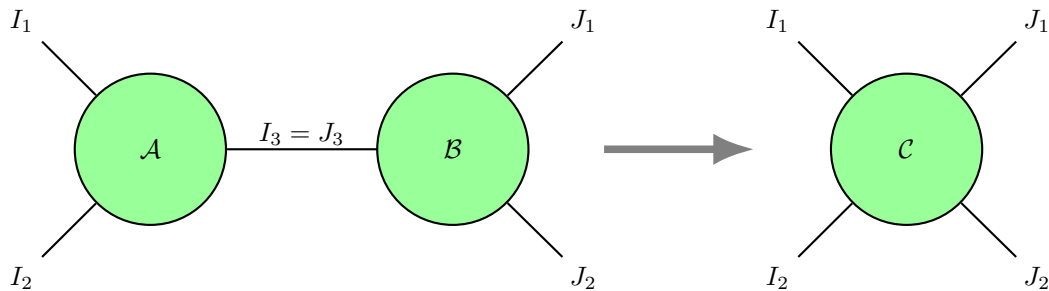


Figure 3: ${}_n \times_m$ -product

Example

Let $\mathbf{A} = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$.

$$\mathbf{A}_{1 \times 1} \mathbf{b} = \begin{bmatrix} 1(5) + 2(6) \\ 3(5) + 4(6) \end{bmatrix} = \begin{bmatrix} 5 + 12 \\ 15 + 24 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

Matricization

Let $\mathcal{A} \in I_1 \times \dots \times I_N$ be an N -th order tensor. The k -th matricization of $\mathcal{A}$ is a matrix $\mathbf{A}_k \in I_1 I_2, \dots, I_k \times I_{k+1} \times I_{k+2} \times \dots \times I_N$, whose elements are taken column-wise from $\mathcal{A}$. The elements are $\mathbf{A}_{(k)}(i_1, \dots, i_k, i_{k+1}, \dots, i_N) = \mathcal{A}(i_1, \dots, i_k, i_{k+1}, \dots, i_N)$. This operation allows us to collapse a tensor into a matrix. However, this loses information about how the dimensions are related. Another name for it is unfolding.

Example

The slices of A along the 3rd mode are two 4×3 matrices.

$$A_1 = \begin{bmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 13 & 17 & 21 \\ 14 & 18 & 22 \\ 15 & 19 & 23 \\ 16 & 20 & 24 \end{bmatrix}.$$

Let us say we want to matricize along the first mode. This compresses the second and third mode into the columns of a matrix, so we would expect to obtain a matrix $\in \mathbb{R}^{4 \times 6}$. We will do this by moving the values in the third mode into the columns.

The notation for matricization along the first mode is $A_{(1)}$.

$$A_{(1)} = \begin{bmatrix} 1 & 5 & 9 & 13 & 17 & 21 \\ 2 & 6 & 10 & 14 & 18 & 22 \\ 3 & 7 & 11 & 15 & 19 & 23 \\ 4 & 8 & 12 & 16 & 20 & 24 \end{bmatrix}$$

Introduce the tensor notation and modeling ideas needed by the reader. Compare the package with the closest existing R software and cite relevant literature using Pandoc citations, for example (R Core Team, 2026).

3 Package design

The main goal of `tensortools` is to provide functions for random number generation, density computation, and MLE estimation for common tensor variate distributions. Additionally, the package provides diagnostic tests for the structure of input data.

One intentional decision was to store independent, identically distributed draws from a tensor variate distribution as a list. For example, if we generate $n = 100$ draws from a tensor variate normal distribution of dimension $2 \times 3 \times 4$, the resulting R object will be a list of length 100. Each element in the list is a base R array object of dimension $2 \times 3 \times 4$. Other packages in this space created unique S3 and S4 class objects to store these types of draws. However, these choices are difficult for the average R user, as they have unique interactions and functions.

Another option would be to output a tensor of size $100 \times 2 \times 3 \times 4$; that is, create a tensor of one higher dimension where the first mode represents the index of the draw. The choice to have the first mode represent the draw is arbitrary and could potentially confuse users, especially if they permute the dimensions.

```
three_dim_iid <- rtnorm(n = 100, mu = array(1:12, dim = c(2, 3, 4)))
```

```
length(three_dim_iid)
#> [1] 100
str(three_dim_iid[1:2], max.level = 1)
#> List of 2
#> $ : num [1:2, 1:3, 1:4] 0.628 1.574 2.341 5.367 3.288 ...
#> $ : num [1:2, 1:3, 1:4] 3.466 0.453 2.369 3.521 5.443 ...
#> - attr(*, "dim")= int 2
```

Describe the package's data structures, model classes, and computational approach. Emphasize design decisions that matter to an R user.

Core workflow

This package contains random number generations for several tensor variate distributions. Below is a simulation from a tensor variate normal of size $2 \times 3 \times 4$. The dimension of the draws is the same as the `mu` object provided. The covariance structure is specified with a list of matrices, where each matrix must be a square of the corresponding mode.

```
mu_true <- array(1:24, dim = c(2, 3, 4))

S1 <- crossprod(matrix(rnorm(4), nrow = 2))
S2 <- crossprod(matrix(rnorm(9), nrow = 3))
S3 <- crossprod(matrix(rnorm(16), nrow = 4))

sigmas_true <- list(S1, S2, S3)

tvn_draws <- rtnorm(n = 1e3, mu = mu_true,
                   sigmas = sigmas_true)

tvn_draws[[1]]
#> , , 1
#>
#>      [,1]      [,2]      [,3]
#> [1,] -0.8306809 -2.418453  5.479902
#> [2,] -0.2631556  7.507547 14.259621
#>
#> , , 2
#>
#>      [,1]      [,2]      [,3]
#> [1,]  7.363271  8.269422  9.313321
#> [2,]  7.999476  6.531726 10.987394
#>
#> , , 3
#>
#>      [,1]      [,2]      [,3]
#> [1,] 13.85713 15.04890 18.22031
#> [2,] 14.42603 12.37154 14.88463
#>
#> , , 4
#>
#>      [,1]      [,2]      [,3]
#> [1,] 19.58052 20.01649 21.83116
#> [2,] 20.30362 19.01035 22.62279
```

Given a list of draws, the package can estimate the MLEs using the function `tensor_mle()`. The `model` argument is required to know which distribution to fit to the data.

```
tvn_mles <- tensor_mle(tvn_draws, model = "normal")

mean((tvn_mles$mu - mu_true)^2)
#> [1] 0.06625785

for(i in 1:3) {
  true_scaled <- sigmas_true[[i]] / sum(diag(sigmas_true[[i]]))
  est_scaled <- tvn_mles$sigmas[[i]] / sum(diag(tvn_mles$sigmas[[i]]))

  print(mean((est_scaled - true_scaled)^2))
}
#> [1] 5.771159e-06
#> [1] 7.186356e-06
#> [1] 1.218794e-05
```

There are also density functions for each distribution. For the tensor variate normal, the required arguments are `mu` and `sigmas`. There is an optional `log` argument to compute the log density rather than the density itself.

```
dtnorm(tvn_draws[[1]], mu = mu_true, sigmas = sigmas_true)
#> [1] 4.654064e-21
```

Show the smallest useful end-to-end example. Keep examples reproducible and fast enough that the full article can be rebuilt in under ten minutes.

4 Model fitting and inference

One major functionality that `tensoRmodels` provides is MLE estimation. This is done via two different algorithms: for the tensor variate normal distribution, a flip-flop algorithm is used. For tensor variate skewed distributions, an EM algorithm is used.

By default, the initial values for the parameters are chosen in different ways. The location parameter $\mathcal{M}$ is started as the median of the draws. This is because the mean of the draws is dependent on the skew and parameters. Each covariance matrix is initialized by centering by the median, matricizing along the corresponding dimension, and adding up the scatter matrix across all of the draws. This total is normalized to have a trace equal to its dimension to ensure the scale is not disproportionate. Initial values for the parameters are set to be conservative and specific to the distribution. For example, for the skewed t , $\nu = 20$. Finally, the skew parameter $\mathcal{A}$ is computed as a function of the expected value of the draws, $\mathcal{M}$, and the relevant parameters for that distribution.

Convergence checks are done by comparing the change in likelihood relative to the previous iteration. At the t th iteration, the equation used is

$$A = \frac{\loglik_t - \loglik_{t-1}}{\loglik_{t-1}}$$

The algorithm ends when $A < \epsilon$, and by default $\epsilon = 1e - 6$.

Explain estimation, initialization, convergence checks, uncertainty quantification, and any constraints or identifiability conventions.

5 Diagnostic Tests

We assess tensor variate normality using the Mahalanobis distance. If $\mathcal{X}$ follows an order- O tensor variate normal distribution, then $\text{vec}(\mathcal{X})$ follows a multivariate normal distribution with mean $\text{vec}(\mathcal{M})$ and covariance matrix $\Sigma_O \otimes \dots \otimes \Sigma_1$. This is why we can vectorize the tensor draws and compare the multivariate Mahalanobis distances to the tensor variate Mahalanobis distances. If we have properly reconstructed the structure using the MLEs $\hat{\mathcal{M}}$ and $\hat{\Sigma}_O, \dots, \hat{\Sigma}_1$, then the standardized draws should be from a standard normal.

If a tensor normal covariance structure is present, then the multivariate and tensor variate distances should be close to one another. Following the matrix variate normality paper, this can be visualized with a distance-distance plot: when the tensor normal structure is appropriate, the distances lie close to the reference line; when the Kronecker product structure is absent, the distances diverge.

The function `mahalanobis_dist()` computes both distances using fitted MLEs, and `mahalanobis_test()` performs a two-sample Kolmogorov-Smirnov test to compare their empirical distributions.

```
S1 <- crossprod(matrix(data = c(1, 0.5, 0.5, 1), nrow = 2))

multivar_draws <- rtnorm(n = 1000, mu = c(2, 3), sigmas = list(S1))
matrix_draws <- rtnorm(n = 1e3, mu = matrix(1:6, nrow = 2, ncol = 3))

A <- matrix(rnorm(6^2), 6, 6)
s_vec <- crossprod(A) # positive definite, general covariance

vec_draws <- rtnorm(n = 1e3, mu = 1:6, sigmas = list(s_vec))

matrix_nocovar_draws <- lapply(1:1e3, function(i) {
  array(vec_draws[[i]], dim = c(2, 3))
})

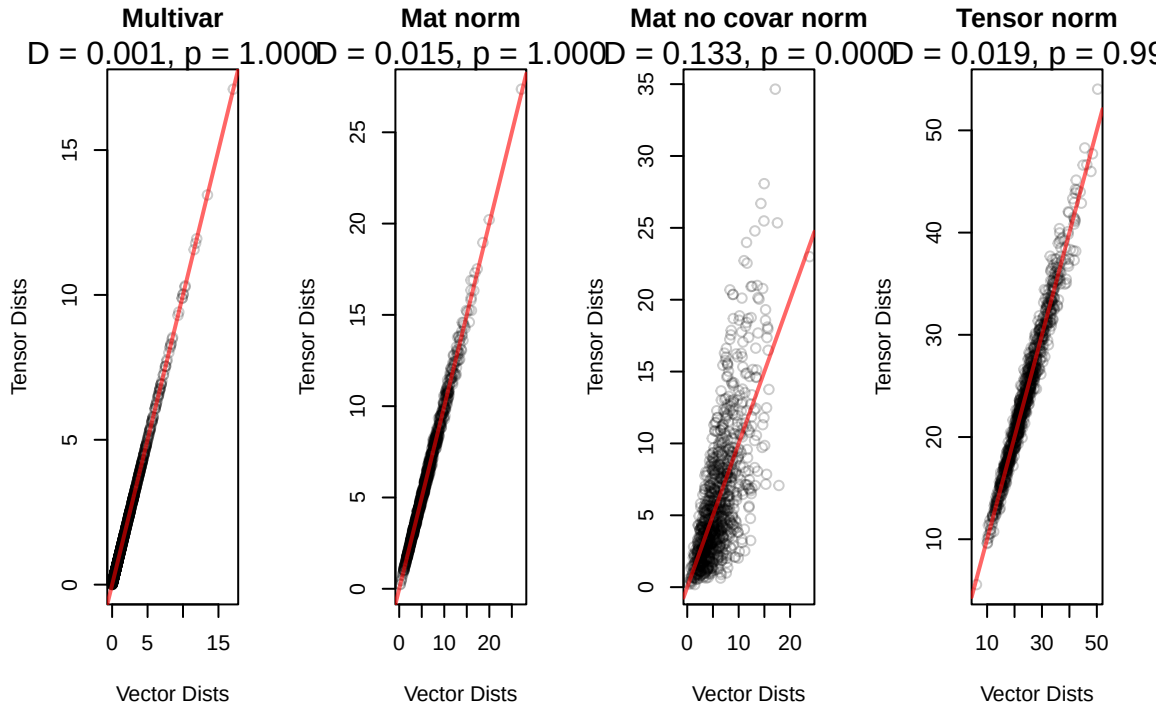
plot_malanobis <- function(draws, title = "Dist") {
  distances <- mahalanobis_dist(draws)
```

```

test <- mahalanobis_test(distances)
plot_dist(distances)
title(main = title)
with(test, mtext(text = sprintf("D = %.3f, p = %.3f", statistic, p.value)),
      side = 3, line = 1)
}

par(mfcol = c(1, 4))
plot_malanobis(multivar_draws, title = "Multivar")
plot_malanobis(matrix_draws, title = "Mat norm")
plot_malanobis(matrix_nocovar_draws, title = "Mat no covar norm")
plot_malanobis(tvn_draws, title = "Tensor norm")

```



We are also interested in performing a likelihood ratio test. We are curious if one of the covariance matrices might be the identity matrix. Thus, we run the algorithm and restrict these matrices.

For a restriction set $\mathcal{R} \subset \{1, \dots, D\}$, the restricted model fixes $\Sigma_r = I_{n_r}$ for every $r \in \mathcal{R}$, while all other covariance matrices are estimated. The likelihood ratio test compares

$$H_0 : \Sigma_r = I_{n_r} \forall r \in \mathcal{R} \quad \text{vs.} \quad H_A : \Sigma_r \neq I_{n_r} \text{ for at least one } r \in \mathcal{R}.$$

The test statistic is

$$2\{\ell(\hat{\theta}_{\text{full}}) - \ell(\hat{\theta}_{\mathcal{R}})\} \sim \chi^2_{\nu_{\mathcal{R}}}, \quad \nu_{\mathcal{R}} = \sum_{r \in \mathcal{R}} \left\{ \frac{n_r(n_r + 1)}{2} - 1 \right\}$$

If the p -value is large, then we fail to reject the null hypothesis and conclude the restricted model fits the data as well as the unrestricted model. If the p -value is small, we reject the null hypothesis, which states the unrestricted model has a significantly better fit than the restricted model.

```

true_str <- kronecker(S3, S2) |> kronecker(S1)

test_restrict <- function(draws, restrict) {
  curr_restrict <- tensor_mle(draws, restrict = restrict, model = "normal")

  if(is.null(restrict)) restrict <- "None"

  curr_str <- with(curr_restrict, kronecker(sigmatas[[3]], sigmas[[2]]) |>
    kronecker(sigmatas[[1]]))
}

```

```

    tibble(mse = mean((curr_str - true_str)^2),
           loglik = curr_restrict$loglik,
           k = curr_restrict$k,
           BIC = curr_restrict$BIC,
           restrict = paste(restrict, collapse = ", "))
  }

(res <- map_dfr(list(NULL, c(1), c(2), c(3), c(1, 2), c(1, 3), c(2, 3)),
               draws = tvn_draws, test_restrict))
#> # A tibble: 7 x 5
#>   mse   loglik    k    BIC restrict
#>   <dbl>   <dbl> <dbl>   <dbl> <chr>
#> 1  70.7 -47371.   41  95026. None
#> 2  68.1 -50061.   39 100391. 1
#> 3  65.2 -62112.   36 124473. 2
#> 4  56.6 -341176.   32 682573. 3
#> 5  64.8 -64846.   34 129927. 1, 2
#> 6  57.5 -349076.   30 698358. 1, 3
#> 7  58.6 -361128.   27 722442. 2, 3

lrt_test <- function(draws, restrict) {
  full_model <- tensor_mle(draws, model = "normal")
  restrict_model <- tensor_mle(draws, model = "normal", restrict = restrict)

  pval <- pchisq(2 * (full_model$loglik - restrict_model$loglik),
                df = full_model$k - restrict_model$k, lower.tail = FALSE)

  tibble(restrict = paste(restrict, collapse = ", "),
         pval = pval)
}

map_dfr(list(NULL, c(1), c(2), c(3), c(1, 2), c(1, 3), c(2, 3)),
        draws = tvn_draws, lrt_test)
#> # A tibble: 7 x 2
#>   restrict pval
#>   <chr>   <dbl>
#> 1 ""      1
#> 2 "1"     0
#> 3 "2"     0
#> 4 "3"     0
#> 5 "1, 2"  0
#> 6 "1, 3"  0
#> 7 "2, 3"  0

true_str <- kronecker(S3, diag(3)) |> kronecker(S1)

second_identity <- rtnorm(n = 1e3, mu = mu_true,
                        sigmas = list(S1, diag(3), S3))

map_dfr(list(NULL, c(1), c(2), c(3), c(1, 2), c(1, 3), c(2, 3)),
        draws = second_identity, test_restrict)
#> # A tibble: 7 x 5
#>   mse   loglik    k    BIC restrict
#>   <dbl>   <dbl> <dbl>   <dbl> <chr>
#> 1 0.00208 -36298.   41  72880. None
#> 2 1.07    -42430.   39  85130. 1
#> 3 0.000832 -36301.   36  72851. 2
#> 4 2.20    -67716.   32 135653. 3
#> 5 1.06    -42433.   34  85102. 1, 2
#> 6 2.41    -73848.   30 147903. 1, 3
#> 7 2.20    -67713.   27 135612. 2, 3

map_dfr(list(NULL, c(1), c(2), c(3), c(1, 2), c(1, 3), c(2, 3)),

```



```

      draws = second_identity, lrt_test)
#> # A tibble: 7 x 2
#>   restrict pval
#>   <chr>    <dbl>
#> 1 ""      1
#> 2 "1"      0
#> 3 "2"     0.342
#> 4 "3"      0
#> 5 "1, 2"   0
#> 6 "1, 3"   0
#> 7 "2, 3"   0

```

6 Performance and validation

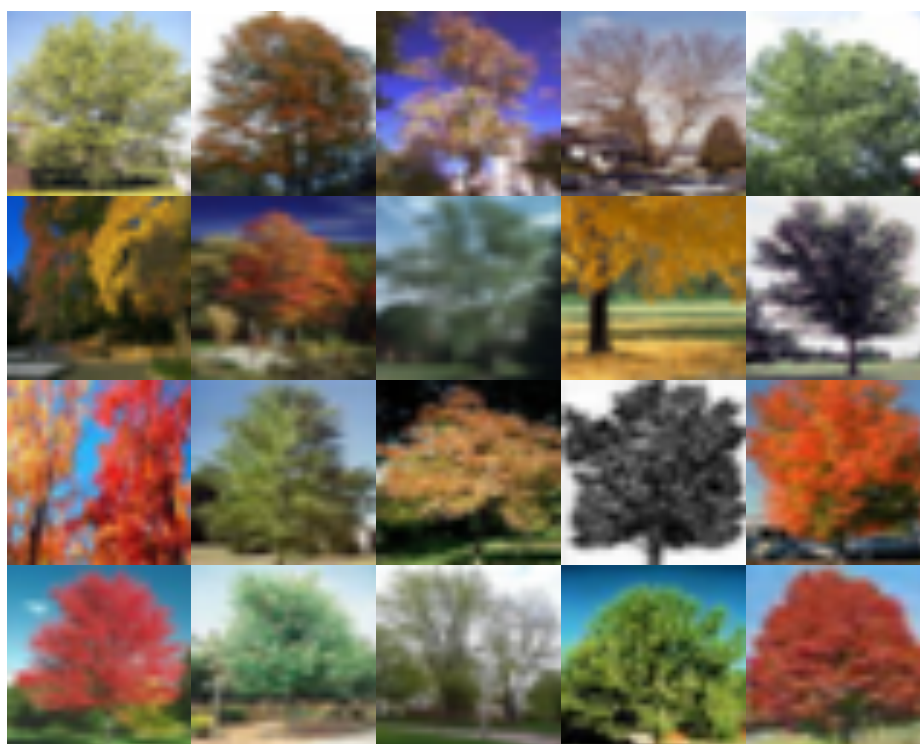
Report numerical validation, simulation results, benchmarks, or comparisons with established implementations. Clearly distinguish package features from claims supported by experiments.

7 Examples

Now that we have set up our tensor distributions, we can use them on a real-world example. The CIFAR 100 dataset contains images of a set of 100 common objects. In this case, we will only be using the maple trees, which have an ID value of 48.

Once we have the CIFAR dataset loaded in from torchvision, we can plot 20 example images. The images are stored as an array of size $32 \times 32 \times 3$. This is an image that is 32×32 pixels, and the last mode represents the red, green, and blue color values.

This is an image that is 32×32 pixels stored as red, green, and blue values. The values vary from 0 to 255 and represent the intensity of that color.



We can fit the best model to this set of images. The function `model_compare()` takes in a dataset and fits all of the possible tensor models. It returns the result as a data frame with columns containing the name of the model, log likelihood, number of parameters denoted as k , AIC, BIC, and the model object itself. We can then select the model with the lowest BIC to proceed.

```
model_compare <- function(draws) {
```

```

res_normal <- tensor_mle(draws, model = "normal", quiet = FALSE)
res_skewt <- tensor_mle(draws, model = "skewt", quiet = FALSE)
res_vargam <- tensor_mle(draws, model = "vargamma", quiet = FALSE)
res_invgauss <- tensor_mle(draws, model = "invgauss", quiet = FALSE)
res_genhyper <- tensor_mle(draws, model = "genhyper", quiet = FALSE)

all_mod <-
  tibble(model = c("Normal", "Skewt", "Vargamma", "Invgauss", "Genhyper"),
    loglik = c(res_normal$loglik, res_skewt$loglik,
      res_vargam$loglik, res_invgauss$loglik, res_genhyper$loglik),
    k = c(res_normal$k, res_skewt$k,
      res_vargam$k, res_invgauss$k, res_genhyper$k),
    AIC = c(res_normal$AIC, res_skewt$AIC,
      res_vargam$AIC, res_invgauss$AIC, res_genhyper$AIC),
    BIC = c(res_normal$BIC, res_skewt$BIC,
      res_vargam$BIC, res_invgauss$BIC, res_genhyper$BIC),
    mod_obj = list(res_normal, res_skewt, res_vargam, res_invgauss, res_genhyper)
  )

all_mod
}

#res <- model_compare(maple_trees)

#res |> select(model, loglik, BIC)

#best_mod <- res$mod_obj[[which.min(res$BIC)]]

best_mod <- tensor_mle(maple_trees, model = "invgauss", quiet = FALSE)

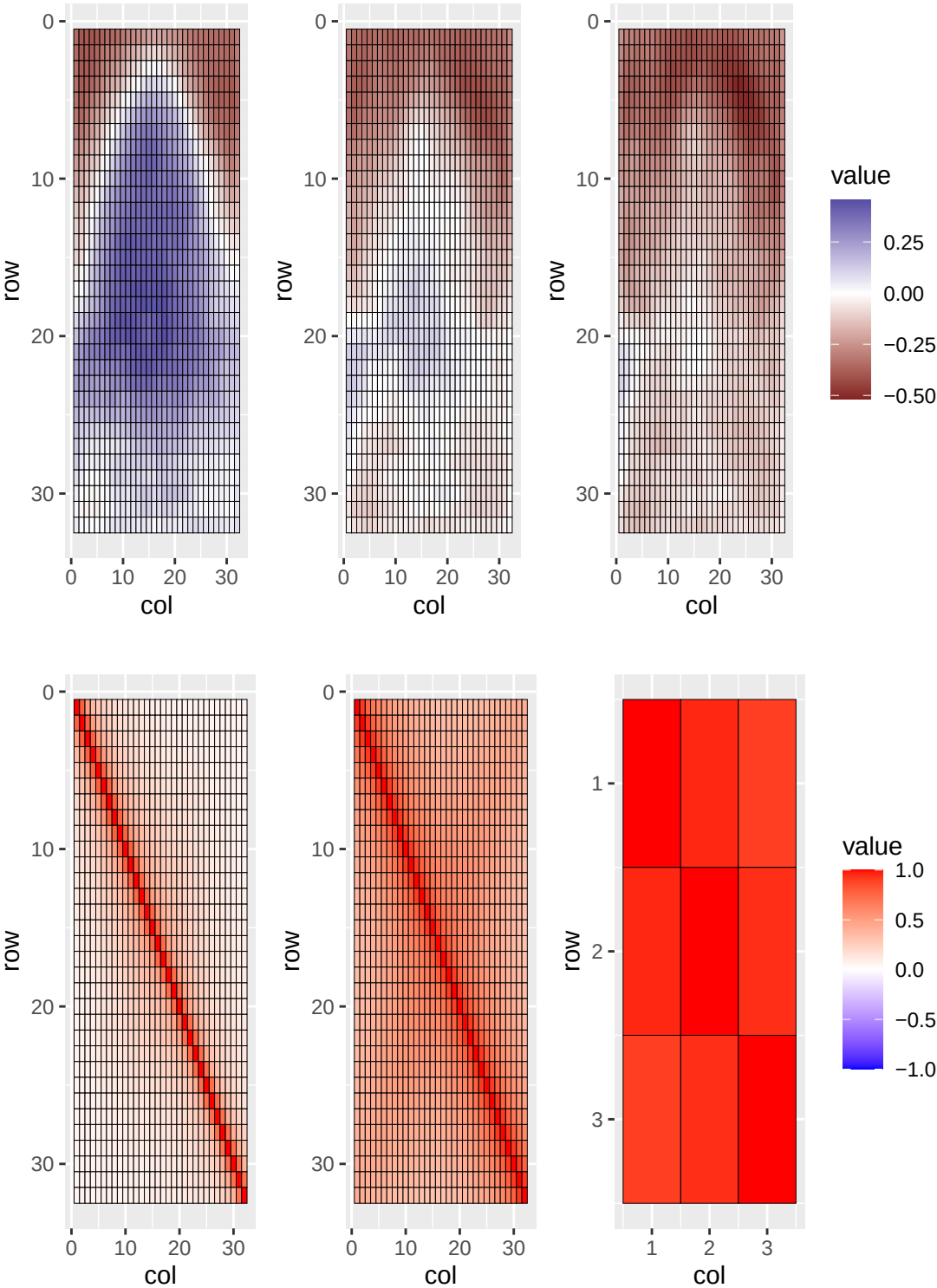
```

In this case, the best model was the normal inverse Gaussian. The expected value of its draws is $\mathcal{M} + \mathcal{A}/\kappa$, where $\mathcal{M}$ is the location parameter and $\mathcal{A}$ is the skew parameter. The plot below contains three images: the first is the empirical average of all of the maple trees without a model assumption. The second is the location parameter $\mathcal{M}$ from the model. Lastly, we have $E[X]$, which is the expected average from the model.

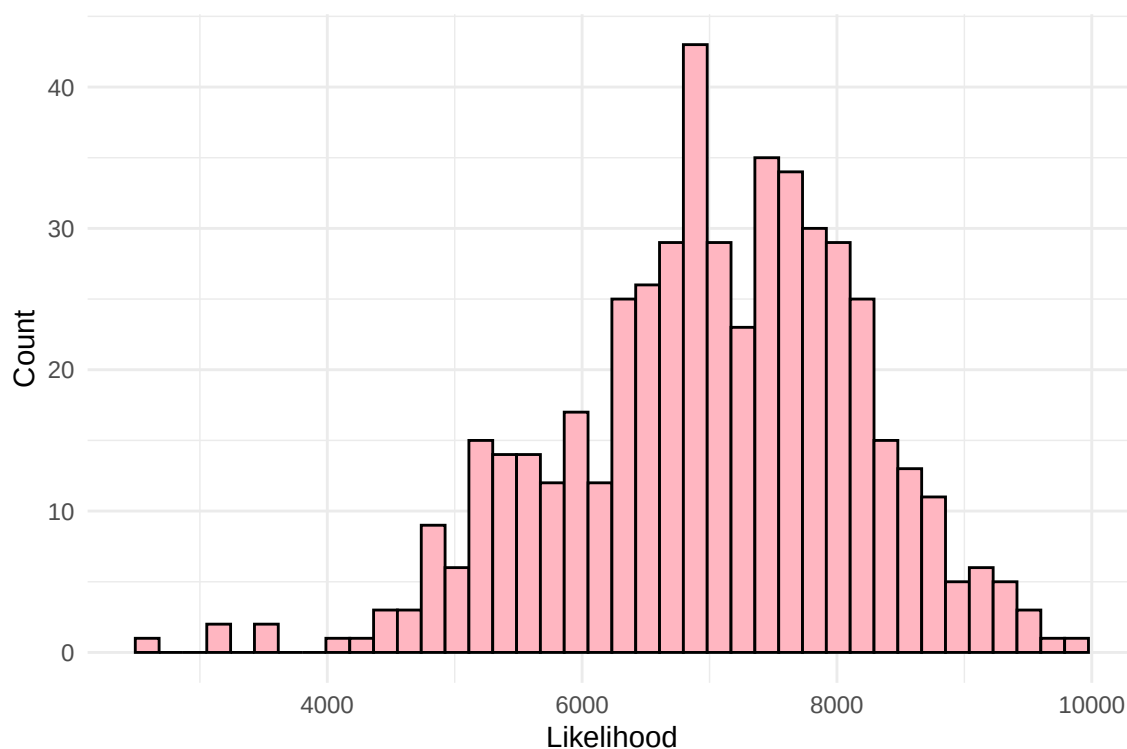
$\mathcal{M}$ is closer to a traditional green maple tree. This seems to be a good starting point to model the variety of colors of the maples, from green to red. Once we add the skew parameter, the $E[X]$ appears similar to the empirical average.



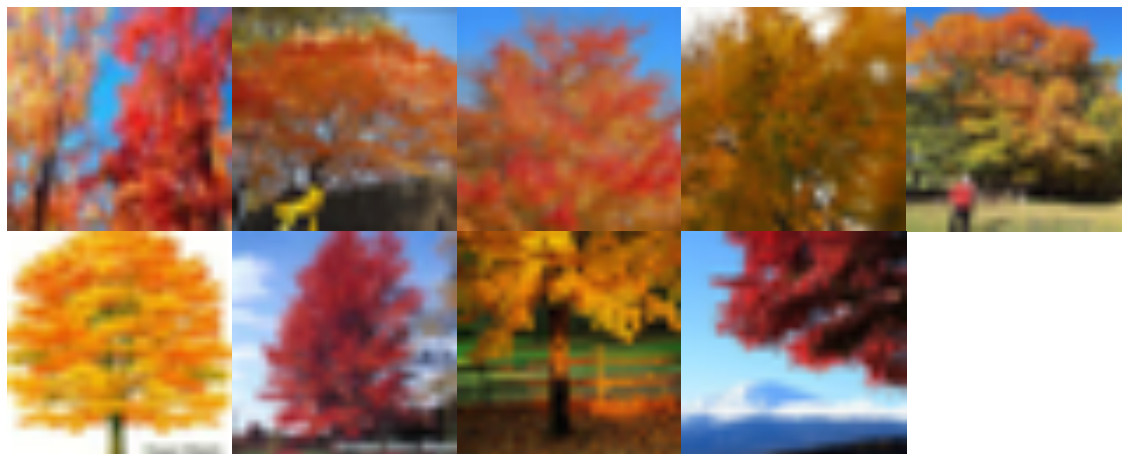
We can also create summary plots of the skew $\mathcal{A}$ and the covariance matrices Σ from the model. We converted the covariance matrices to correlation matrices to make it easier to interpret each matrix, since this places them on the same scale.



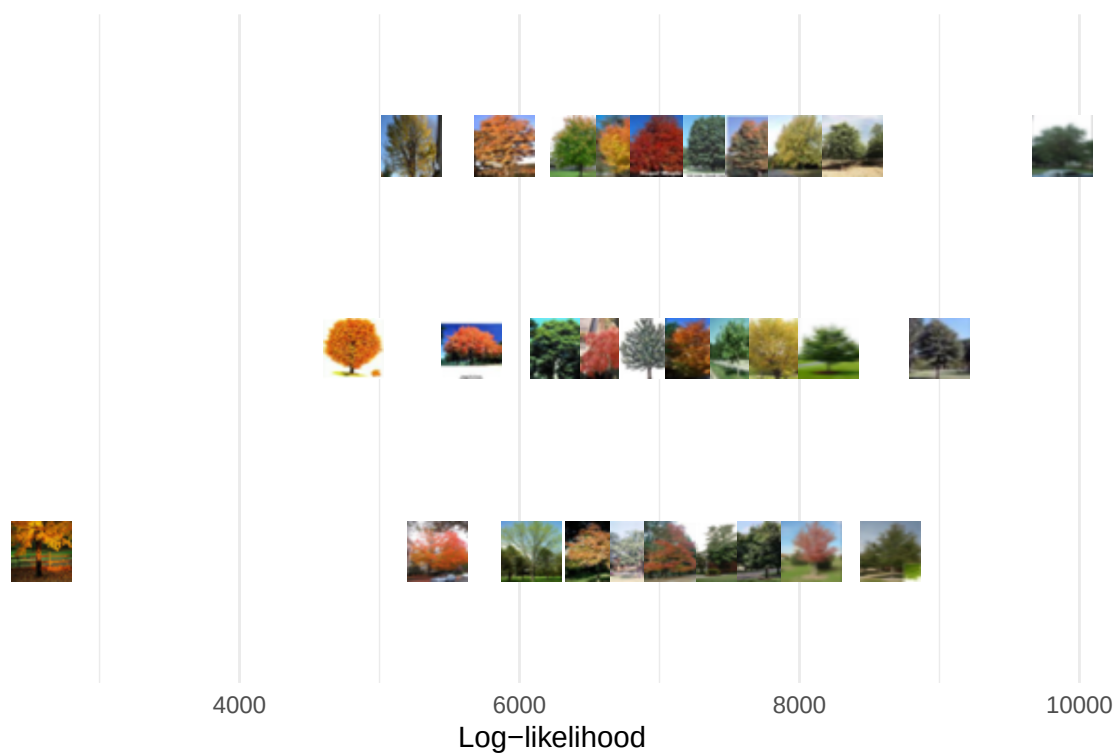
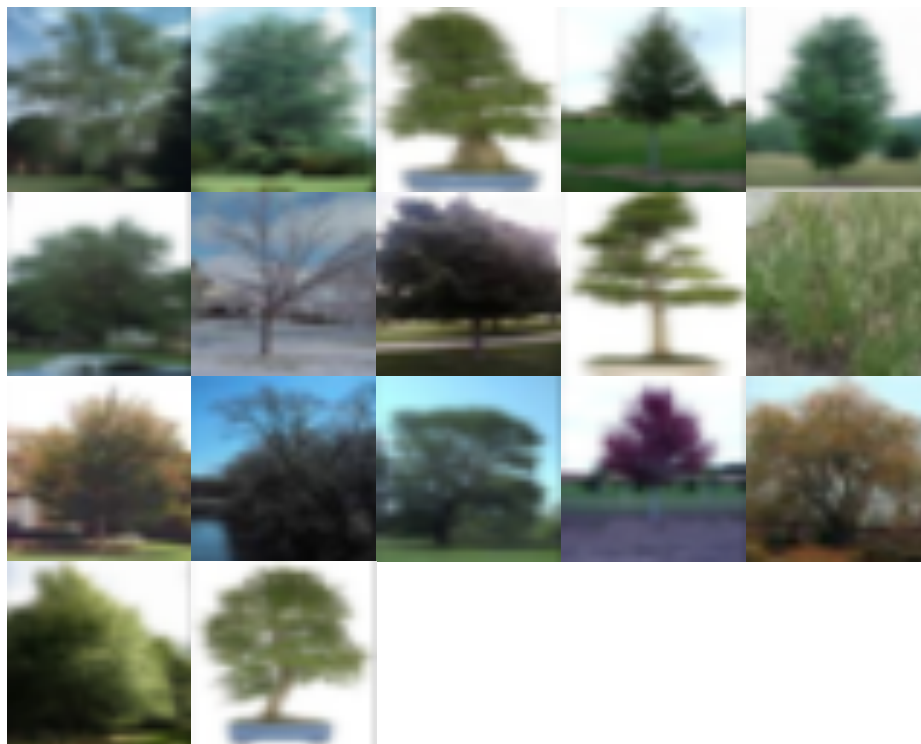
Now that we have the best model for the maple trees, we can obtain the likelihood of each image.



There are clearly some outliers, both at much smaller and much larger likelihoods. Out of curiosity, I plotted the lowest likelihoods. All of these maples are predominantly orange and red, which differ from the average tensor that the model had. It seems that the data contains many more green maples, so the model struggles to account for the possible variation in tree color.



As expected, all of the maples that had a large likelihood follow the pattern of being a forest green and mainly in the center of the image.



Use one or two examples that demonstrate what the package makes possible. Prefer datasets that can be included with the submission or downloaded from a stable public archive.

8 Discussion

Summarize the package's strengths, limitations, and intended scope. Mention important future work without turning this section into a feature wish list.

9 Reproducibility

List any external data and scripts required to reproduce the article. Place supporting code in `scripts/`, data in `data/`, and figure assets in `figures/`.

```
#> R version: 4.5.2  
#> Platform: aarch64-apple-darwin20  
#> rjtools version: 1.0.18.1
```

10 Acknowledgments

Add funding, contributor, and institutional acknowledgments here.

Bibliography

R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2026. URL <https://www.R-project.org/>. [p4]

Anupama Kannan, Dr. David Kahle, Dr. Michael Gallagher
Baylor University
Department of Statistics
Waco, Texas
ORCID: 0000-0003-2789-6997
Anupama_Kannan1@baylor.edu